P6: REPASO

Convención sobrecarga operadores aritméticos

🔷 __add__: Permite sumar otro concesionario (fusionar) o añadir un coche al concesionario.
🔷 __radd__: Permite sumar un coche al concesionario cuando la operación está al revés ("Coche" + concesionario).
• 🔷 __iadd__: Permite modificar el concesionario en su lugar (concesionario += coche).

```
class Concesionario:
    def __init__(self, nombre):
        self.nombre = nombre
        self.coches = []

    def __repr__(self):
        return f"{self.nombre}: {self.coches}"

    def __add__(self, other):
        """ Permite sumar concesionarios o agregar coches """
        if isinstance(other, Concesionario):
            # Fusionar concesionarios
            nuevo = Concesionario(f"{self.nombre} & {other.nombre}")
            nuevo.coches = self.coches + other.coches
            return nuevo
        elif isinstance(other, str):
            # Agregar un coche
            nuevo = Concesionario(self.nombre)
            nuevo.coches = self.coches + [other]
            return nuevo
        else:
            raise TypeError("Solo se pueden sumar concesionarios o agregar
coches como string.")
```

🔹 __add__: Permite sumar otro concesionario (fusionar) o añadir un coche al concesionario.
🔹 __radd__: Permite sumar un coche al concesionario cuando la operación está al revés ("Coche" + concesionario).
• 🔹 __iadd__: Permite modificar el concesionario en su lugar (concesionario += coche).

```
class Concesionario:

    def __radd__(self, other):
        """ Permite sumar un coche al concesionario en orden inverso ('coche'
+ concesionario) """
        if isinstance(other, str):
            return self + other  # Llama a __add__
        else:
            raise TypeError("Solo se pueden agregar coches como string.")

    def __iadd__(self, other):
        """ Permite modificar el concesionario en su lugar (concesionario +=
coche o concesionario += otro concesionario) """
        if isinstance(other, Concesionario):
            self.coches += other.coches  # Fusiona los inventarios
        elif isinstance(other, str):
            self.coches.append(other)  # Añade un coche
        else:
            raise TypeError("Solo se pueden agregar concesionarios o coches
como string.")
        return self
```

```
c1= "Toyota Corolla"
c2= "Honda Civic"
conc=Concesionario("AutoMarket")
conc+=c1 #iadd
conc+=c2 #iadd
print(conc)

conc2=Concesionario("Auto Market2")
c3= "Toyota Corolla 2"
conc+=conc2 #iadd
print(conc)
```

```
c4= "Toyota Corolla 3"
conc = conc+c4 #add
print(conc)

c5= "Toyota Corolla 4"
conc = c5+conc #radd, que internamente llama a add
print(conc)
```

Convención implementación

Sobrecarga de operadores aritméticos +,-,*,/, //…

Por convención, implementaremos estos operadores de manera que add y radd devuelven una nueva instancia de la clase que tenga los cambios requeridos, en lugar de realizar los cambios directamente en nuestra instancia.

iadd es el único que cambia self.

```
#Class concesionario
def __add__(self,coche): #por convención
    new_lcoches = self.lcoches.copy()
    new_lcoches.append(coche)
    return Concesionario(self.nombre, new_lcoches)

def __radd__(self,coche):
    new_lcoches = self.lcoches.copy()
    new_lcoches.insert(0,coche)
    return Concesionario(self.nombre, new_lcoches)

def __iadd__(self,coche): #EXCEPCIÓN
    self.lcoches.append(coche)
    return self
```

Paso ‘por referencia’ o ‘por valor’ de parámetros

Cuando asigno algo a una variable dentro de un método, esa variable será una variable local (aunque se llame igual que un parámetro del método)

Tipos Inmutables: paso 'por valor'

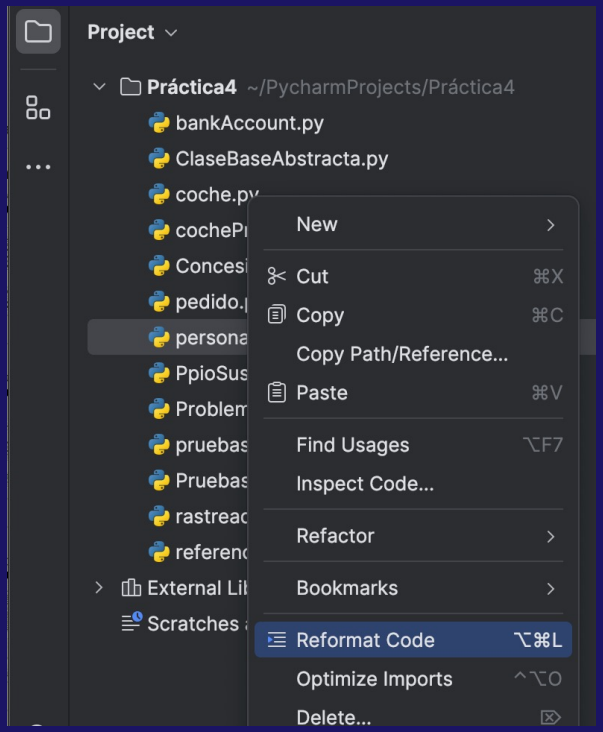
Tipos Mutables: paso 'por referencia'

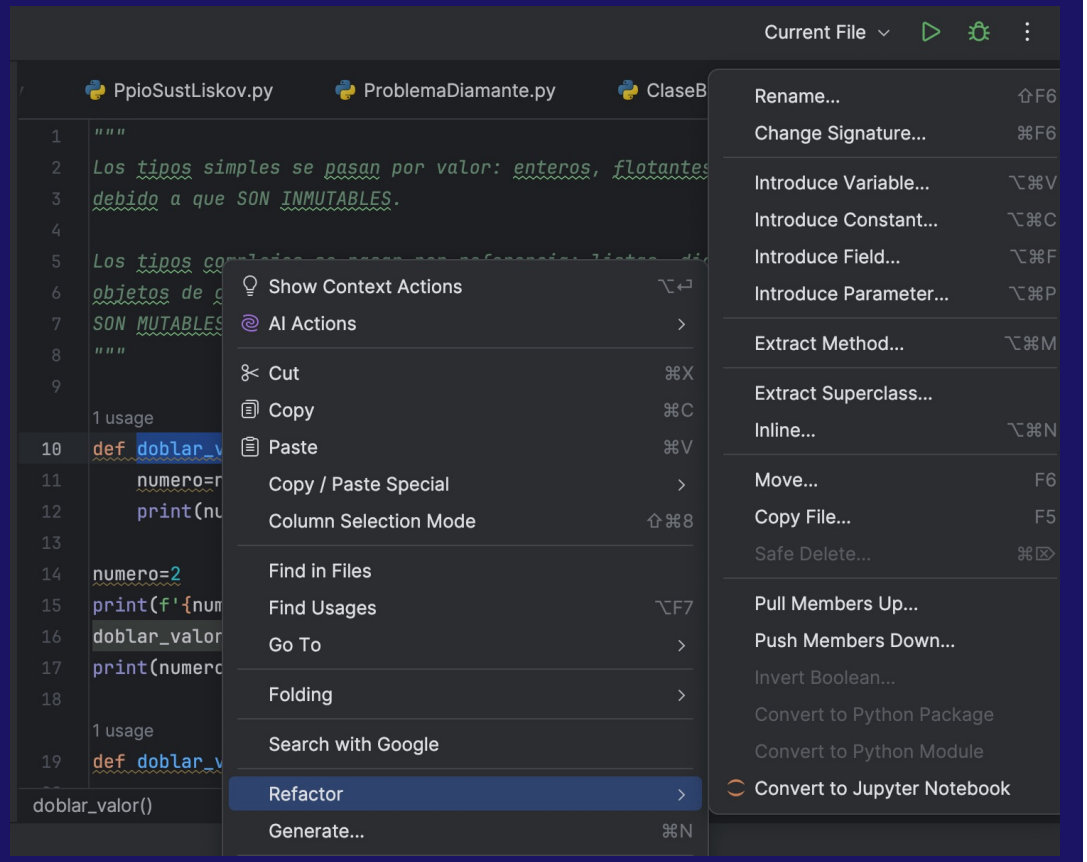
```
def doblar_valor (numero:int)-> None:
    numero=numero*2
    print(numero) #4

numero=2
doblar_valor(numero)
print(numero) #2

def doblar_valores(numeros:list)->None:
    numeros=numeros*2 #por convención, aquí estoy creando una nueva
lista que reasigno a la variable local números, sin cambiar la variable
inicial.
    print(numeros) #lista duplicada

def cambiar_valores(numeros:list)->None:
    for i,n in enumerate(numeros):
        numeros[i]*=2
    print(numeros) #lista con números duplicados

numeros=[2,4,6]
doblar_valores(numeros)
print(numeros)# [2,4,6]
cambiar_valores(numeros)
print(numeros) [4, 8, 12]
```

```
#copia al vuelo de una lista: [:]
Numeros=[2,4,6]
cambiar_valores(numeros[:])
print(numeros)  #[2,4,6]
```

Refactoring

¿Qué es el refactoring?





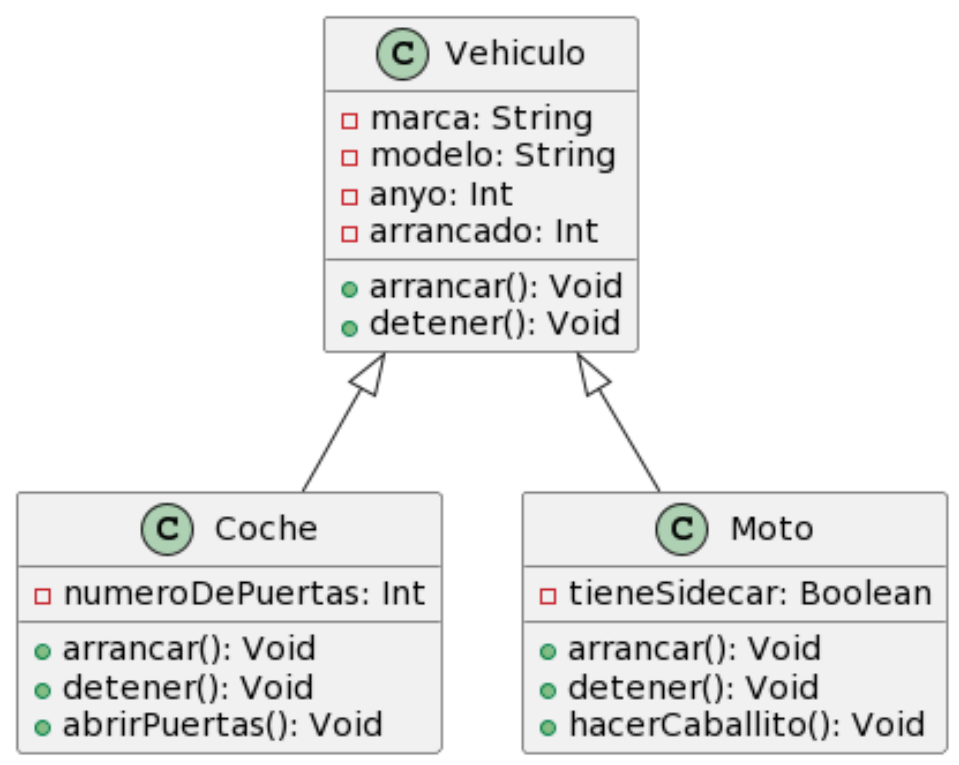
- Reformateo de código (PEP-8)

- Funciones: cambiar signatura, renombrar, extraer método, extraer superclase, …

Herencia simple

Herencia Simple

Métodos con el mismo nombre

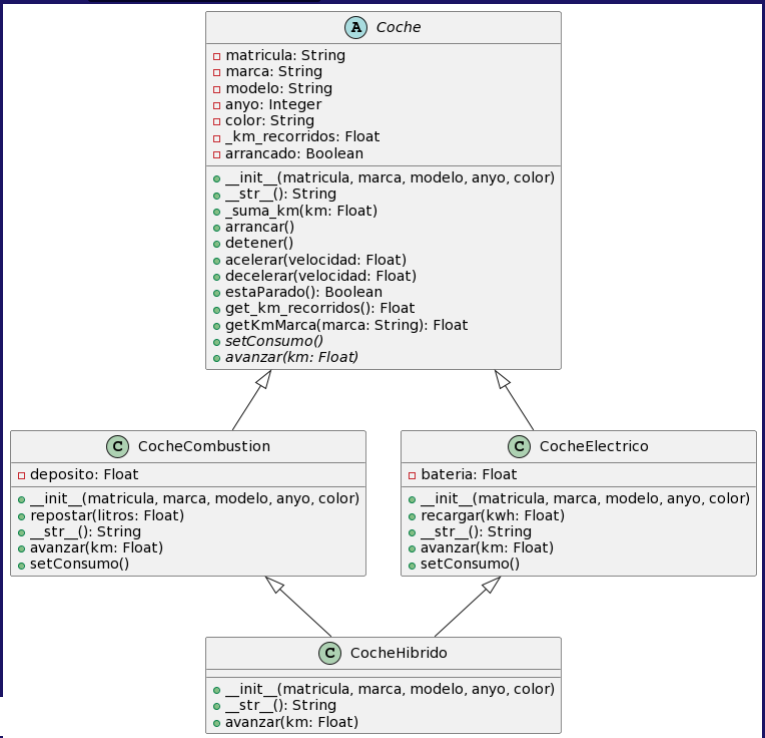


Métodos **extendidos**: la clase hija implementa el mismo método que la clase padre, y dentro del código llama al método del padre (reutiliza el código)

Métodos **sobreescritos**: la clase hija redefine el método y descarta por completo la funcionalidad del padre.

Herencia múltiple: problema diamante

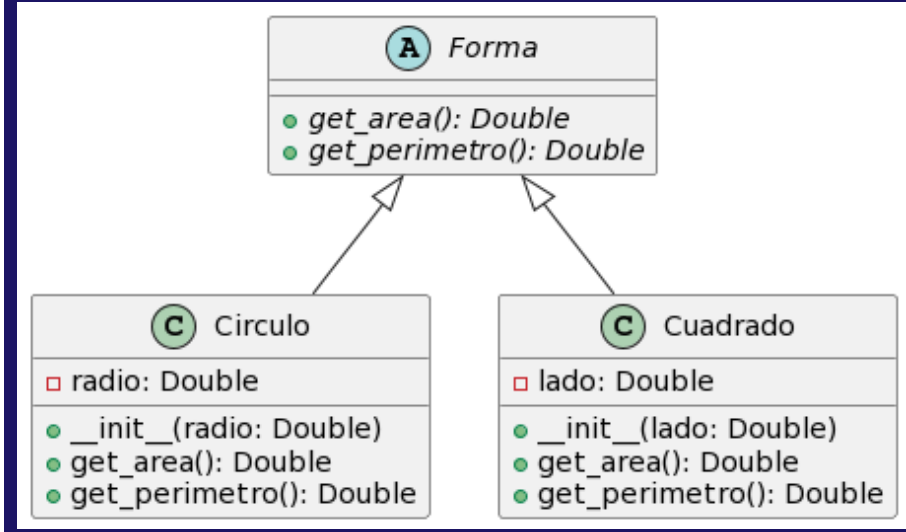
Problema Diamante




```python
class A:
    def metodo(self):
        print("Método de la clase A")

class B(A):
    def metodo(self):
        print("Método de la clase B")  #sobreescribe clase A (si extendiera usaría super()

class C(A):
    def metodo(self):
        print("Método de la clase C") #sobreescribe clase A

class D(B,C):
    pass

# Creación de un objeto de la clase D
d = D()
d.metodo() #¿A quién llamo? MRO: B,C,A
print(D.__mro__) #para ver el orden de resolución
```

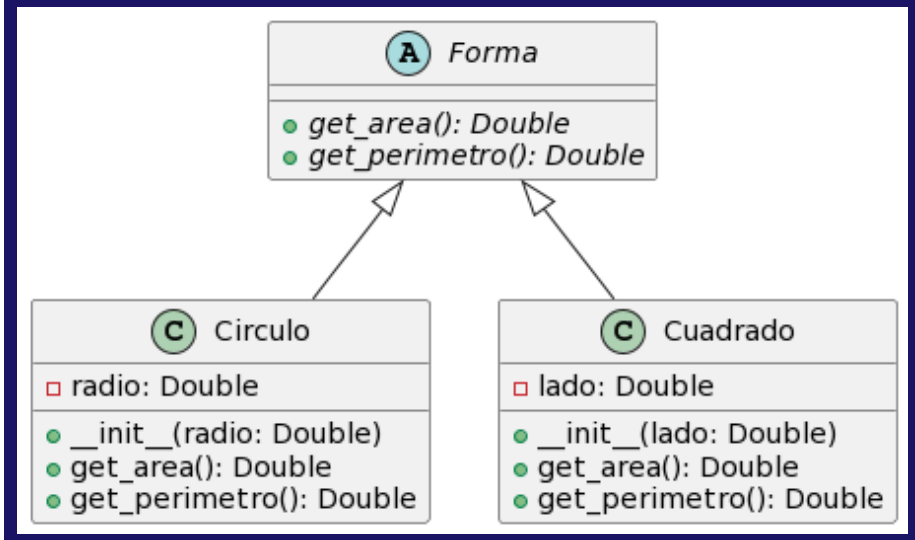
Clases abstractas

Clases Abstractas




```
from abc import ABC, abstractmethod
from math import pi

class Forma(ABC):  #abstract base class: no se puede instanciar. Si todos los métodos
abstractos, estamos ante una interfaz.
    @abstractmethod
    def get_area(self):
        pass
    @abstractmethod
    def get_perimetro(self):
        pass

class Circulo(Forma): #tiene que definir todos los métodos abstractos
    def __init__(self,radio):
        self.radio=radio
    def get_area(self):
        return pi * (self.radio**2)
    def get_perimetro(self):
        return 2 * pi * self.radio

class Cuadrado(Forma):
    def __init__(self,lado):
        self.lado=lado
    def get_area(self):
        return self.lado**2
    def get_perimetro(self):
        return 4*self.lado

formas=[Circulo(10),Circulo(20),Cuadrado(2),Cuadrado(4)]
for forma in formas:
    print(f'{type(forma).__name__}. Area: {forma.get_area()}. Perimetro:
```

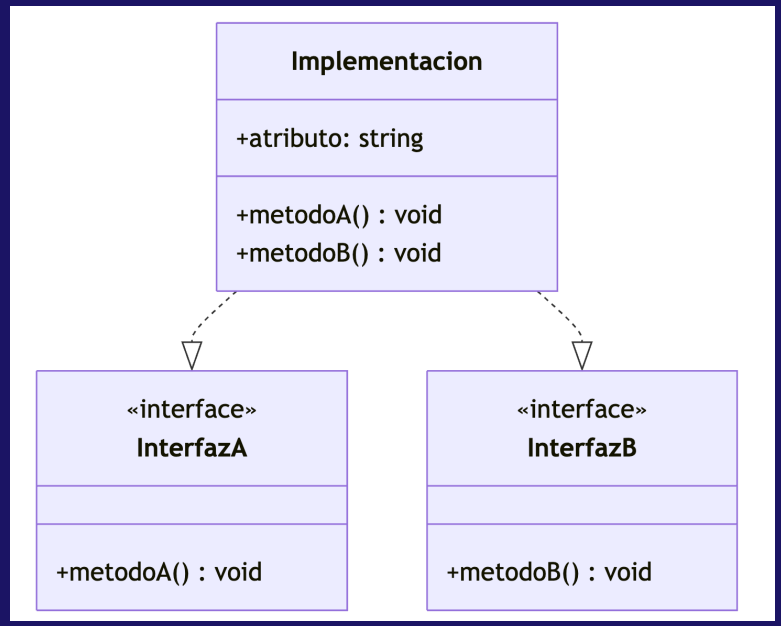
Clases abstractas



- Cuando un método se decora como abstracto, debe definirse obligatoriamente en cualquier clase hija, y evita que la clase que lo contiene pueda instanciarse

- Si en la clase hija no se redefine ese método, la clase hija tampoco podrá ser instanciada

Interfaces (Simulación)



Python no define interfaces como tales.

Una interfaz en Python se implementa como una clase abstracta que **solo define qué métodos deben implementarse**, sin proporcionar ninguna implementación. Normalmente, tampoco tiene atributos (aunque técnicamente pueden agregarse).

Las subclases deben implementar **todos los métodos de la 'interfaz'**.

Característica	Clase Abstracta	Interfaz (Simulada)
Métodos concretos	**Sí**, puede tenerlos	**No**, solo define métodos
Atributos	**Sí**, puede tenerlos	**No**, usualmente no tiene
Propósito	Base para herencia y reutilización de código	Definir un contrato que deben seguir las clases
Uso en Python	`ABC` con métodos abstractos y concretos	`ABC` con solo métodos abstractos o `Protocol`

Atributos protegidos y privados

Atributos ‘protegidos’

Python no tiene una verdadera privacidad como en otros lenguajes como Java o C++, pero permite indicar que un atributo es protegido o privado usando un guion bajo (_) o doble guion bajo (__) al comienzo del nombre.

```
class Ejemplo:
    def __init__(self):
        self._protegido = "Este es un atributo protegido"

obj = Ejemplo()
print(obj._protegido)  # ⚠️ Se puede acceder, pero no se recomienda
```

Herencia: Son **accesibles** en las subclases.

Atributos ‘privados’

Con los atributos que comienzan por doble subrayado, Python hace *name mangling*, cambiando internamente el nombre del atributo a _Ejemplo__privado para evitar colisiones en clases derivadas. **No es totalmente inaccesible**, solo es más difícil de modificar accidentalmente.

```
class Ejemplo2:
    def __init__(self):
        self.__privado = "Este es un atributo privado"

obj = Ejemplo2()
print(obj.__privado)  # ❌ AttributeError: 'Ejemplo' object has no attribute '__privado’
print(obj._Ejemplo2__privado)  # ✅ Funciona, pero no es buena práctica
```

Herencia. **No son accesibles directamente** en la subclase debido al *name mangling*. Python los renombra internamente (_ClasePadre__atributo), lo que impide el acceso directo en subclases.
Solución: acceder mediante los métodos getter y setter del padre
Nota: se puede acceder también a ellos con el nombre ‘mangleado’, pero no es buena práctica, porque si cambia el nombre de la clase padre, el código dejará de funcionar.

¿Cómo acceder correctamente a atributos ‘privados’?

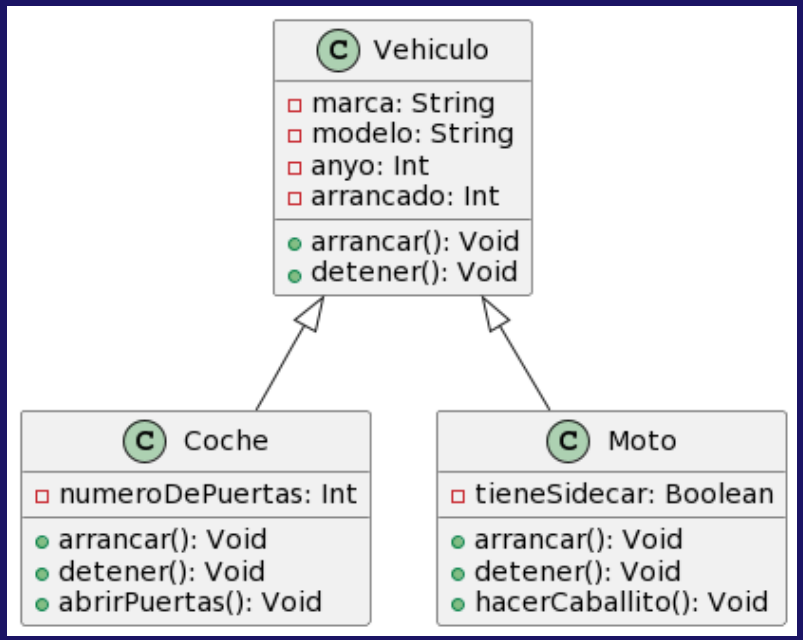
Se usa **encapsulación** con *getters* y *setters*:

```
class Ejemplo:
    def __init__(self):
        self.__privado = "Dato privado"

    def get_privado(self):
        return self.__privado

    def set_privado(self, valor):
        self.__privado = valor

obj = Ejemplo()
print(obj.get_privado())  # ✅ Acceso seguro
obj.set_privado("Nuevo valor")
print(obj.get_privado())  # Nuevo valor
```

Proyectos: Principios SOLID de Diseño OO

Principio sustitución Liskow

Herencia: Principio Sustitución de Liskov

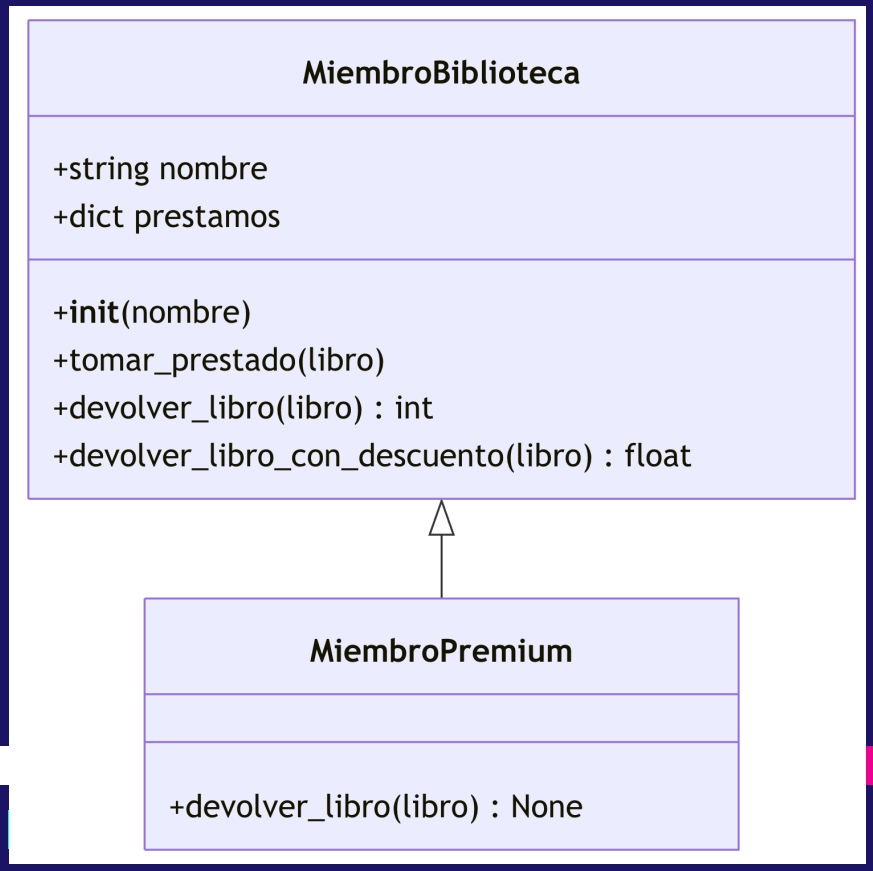


Principio de sustitución de Liskov: Asegúrate de que los métodos que sobreescribes en los hijos son siempre más **restrictivos** que los métodos que defines en los padres.

De esta manera, en cualquier sitio donde paso como parámetro un objeto de tipo padre, podré pasar un objeto de tipo hijo sin problemas. Recuerda: un hijo ES un padre.

Preservar este principio ayuda a crear diseños robustos y flexibles.

¿Se cumple aquí?



En una **biblioteca**, hay dos tipos de usuarios:

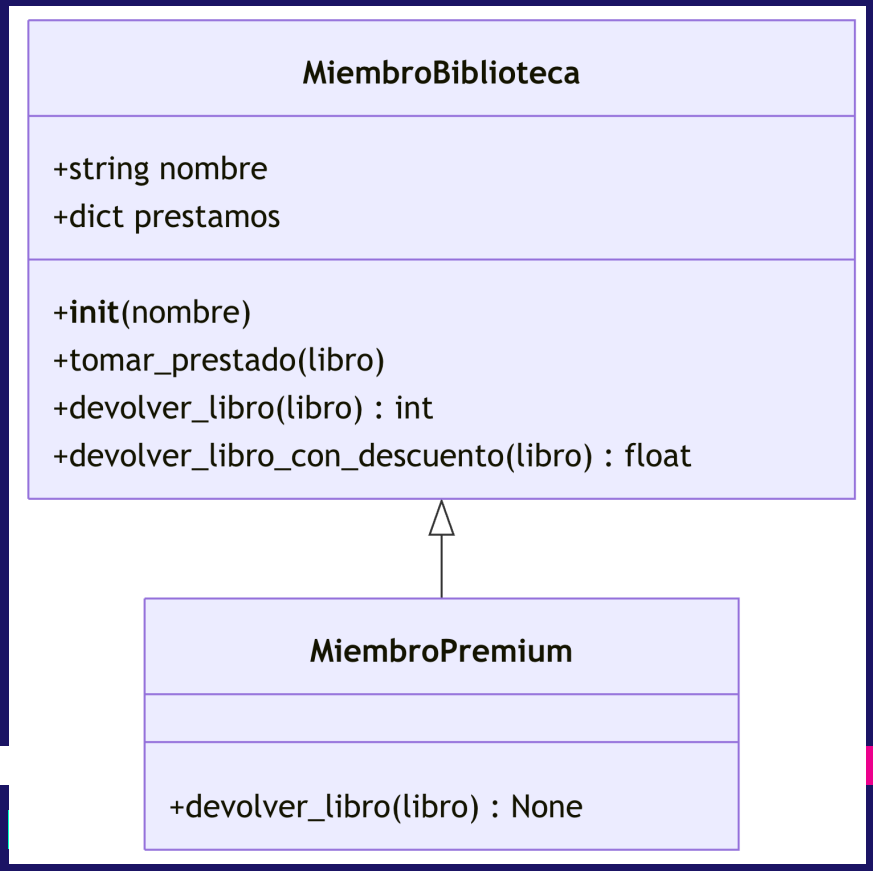
- **Miembro normal (MiembroBiblioteca)**: atributos nombre, número de socio y lista de préstamos actuales. Puede tomar prestado un libro y puede devolverlo. Restricción: debe devolverlo en **7 días**. Si se retrasa, recibe **una multa (retorno método devolver_libro)**.

- **Miembro premium (MiembroPremium)**: tiene **un mes de plazo** para devolver el libro y **nunca recibe multas**. Es por tanto **menos restrictivo** que el padre. El método devolver_libro ahora devuelve None

📌 **El problema surge cuando tratamos MiembroPremium como un MiembroBiblioteca y el sistema espera que se apliquen multas automáticamente en caso de retraso.**

📌 **Esto rompe el Principio de Sustitución de Liskov (LSP) porque MiembroPremium altera las reglas de penalización esperadas en MiembroBiblioteca.**

¿Se cumple aquí?



Imaginad que se extiende MiembroBiblioteca con la siguiente función:

```
def calcular_descuento_en_multa(miembro: Miembro, libro: str):
    """Aplica un descuento del 50% a la multa de los miembros"""
    multa = miembro.devolver_libro(libro)  # Se asume que este método devuelve la multa
    return multa * 0.5  # Se espera que haya una multa para poder calcular el descuento
```

Este método automáticamente lo hereda el hijo, que ahora puede hacer:

```
maria = MiembroPremium("Maria")
print("\n🔹 Código polimórfico: heredo devolver_libro_con_descuento, que espera que haya una multa")
maria.tomar_prestado("El Principito")
maria.prestamos["El Principito"] -= timedelta(days=10)  # Simulamos un retraso de 10 días
multa=maria.devolver_libro_con_descuento("El Principito")  # ❌ No se aplica multa, rompe Liskov
```

Single Responsibility Principle

Una clase debe tener una única razón para cambiar

```
# ❌ Mal diseño: Una sola clase hace varias cosas
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def calculate_pay(self):
        return self.salary * 1.10  # Suponiendo un
bono del 10%

    def save_to_database(self):
        print(f"Saving {self.name} to database")
```

```
# ✅ Buen diseño: Separación de responsabilidades
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def calculate_pay(self):
        return self.salary * 1.10

class EmployeeRepository:
    def save_to_database(self, employee):
        print(f"Saving {employee.name} to database")
```

Principio Abierto/Cerrado

Las clases deben estar abiertas para la extensión, pero cerradas para la modificación

```
# ❌ Mal diseño: Modificar la clase original cada vez que agregamos un nuevo tipo de pago
class Employee:
    def __init__(self, name, salary, type):
        self.name = name
        self.salary = salary
        self.type = type

    def calculate_pay(self):
        if self.type == "full-time":
            return self.salary * 1.10
        elif self.type == "contract":
            return self.salary * 1.05
        else:
            return self.salary
```

```
# ✅ Buen diseño: Extender la funcionalidad mediante herencia
from abc import ABC, abstractmethod
class Employee(ABC):
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    @abstractmethod
    def calculate_pay(self):
        pass

class FullTimeEmployee(Employee):
    def calculate_pay(self):
        return self.salary * 1.10

class ContractEmployee(Employee):
    def calculate_pay(self):
        return self.salary * 1.05
```

Principio de segregación de interfaces

Los clientes no deberían verse forzados a depender de interfaces que no usan

```
# ❌ Mal diseño: Forzamos a todos los trabajadores a implementar métodos innecesarios
class Worker:
    def work(self):
        pass
    def eat(self):
        pass

class Developer(Worker):
    def work(self):
        print("Coding")

    def eat(self):
        raise NotImplementedError("Developers don't take breaks 😆")  # 🚨 Problema
```

```
# ✅ Buen diseño: Interfaces más específicas
class Workable:
    def work(self):
        pass
class Eatable:
    def eat(self):
        pass

class Developer(Workable):
    def work(self):
        print("Coding")

class OfficeWorker(Workable, Eatable):
    def work(self):
        print("Working in the office")
    def eat(self):
        print("Eating lunch")
```

Principio de Inversión de Dependencia

Los módulos de alto nivel no deberían depender de módulos de bajo nivel; ambos deben depender de abstracciones

```
# ❌ Mal diseño: Alta dependencia de la clase concreta MySQLDatabase
class MySQLDatabase:
    def connect(self):
        print("Connecting to MySQL")

class UserService:
    def __init__(self):
        self.db = MySQLDatabase()  # 🚨 Dependencia fuerte

    def get_user(self):
        self.db.connect()
        print("Getting user")
```

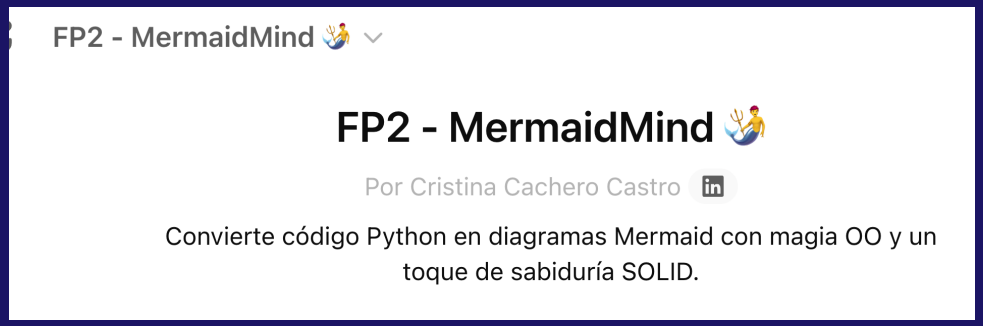
```
# ✅ Buen diseño: Inversión de dependencias usando abstracción
class Database(ABC):
    @abstractmethod
    def connect(self):
        pass
class MySQLDatabase(Database):
    def connect(self):
        print("Connecting to MySQL")
class UserService:
    def __init__(self, db: Database):  # Inyección de dependencia
        self.db = db
    def get_user(self):
        self.db.connect()
        print("Getting user")
# podemos usar diferentes bases de datos sin cambiar UserService
db = MySQLDatabase()
service = UserService(db)
service.get_user()
```

Proyectos: Asistente MermaidMind

https://chatgpt.com/g/g-67b7609b80c88191992799fca581ea77-fp2-mermaidmind

Objetivo

- Generar especificaciones de diseño (diagramas de clases) en lenguaje Mermaid del sistema descrito por el usuario
- Tiene instrucciones para que los sistemas generados cumplan los principios SOLID de diseño OO, así como otras buenas prácticas de diseño reconocidas como los patrones GRASP de Craig Larman.



Sugerencias de uso

- Subidle vuestra descripción textual del sistema y pedidle que os genere la especificación Mermaid de dicho sistema. Visualizadlo en Mermaid Live Editor (https://mermaid.live/).
- Subidle el código Python de todas las clases y pedidle que os evalúe la calidad de vuestro diseño, y os sugiera posibles cambios para hacerlo más robusto